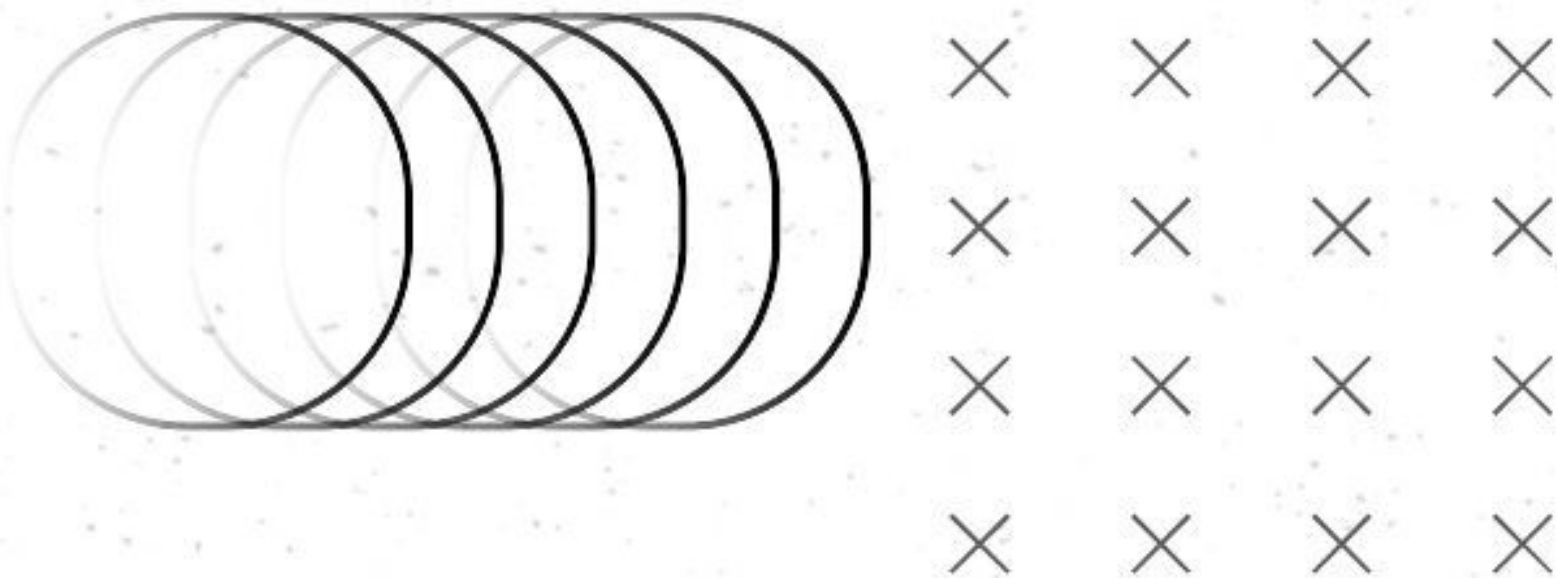
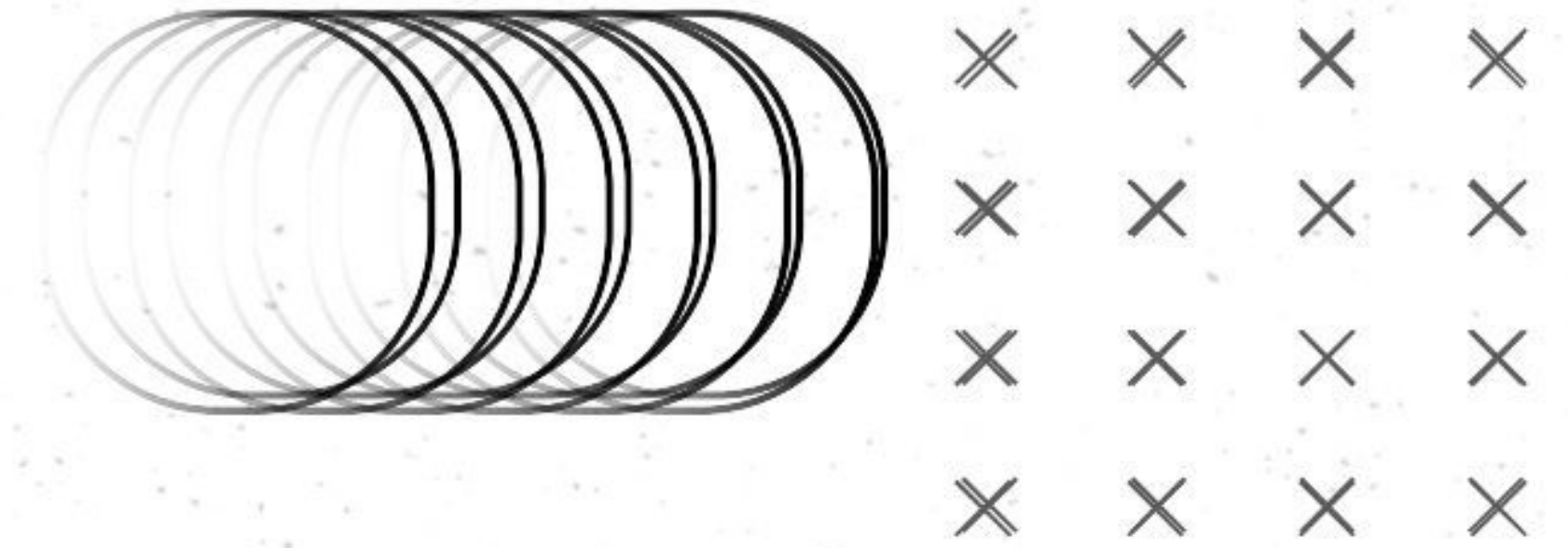








# DATA STRUCTURE

LECTURE " 2 " : Linear Structures:  
Arrays, Stacks, and Queues





# WHAT IS AN ARRAY?

**AN ARRAY IS A COLLECTION OF ELEMENTS STORED IN ORDER  
AND ACCESSED BY INDEX.**

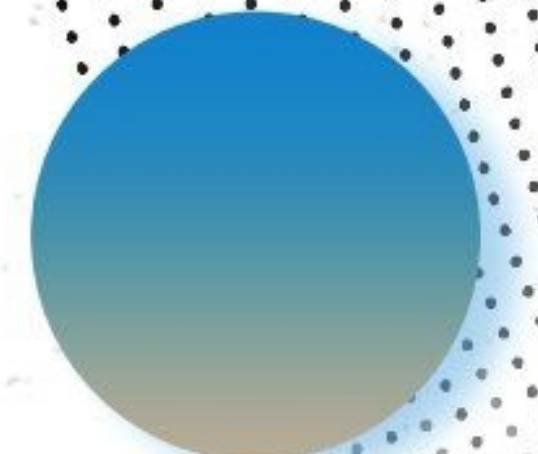
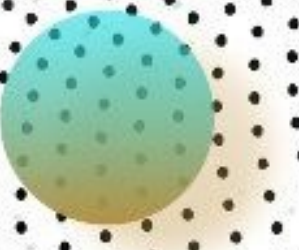
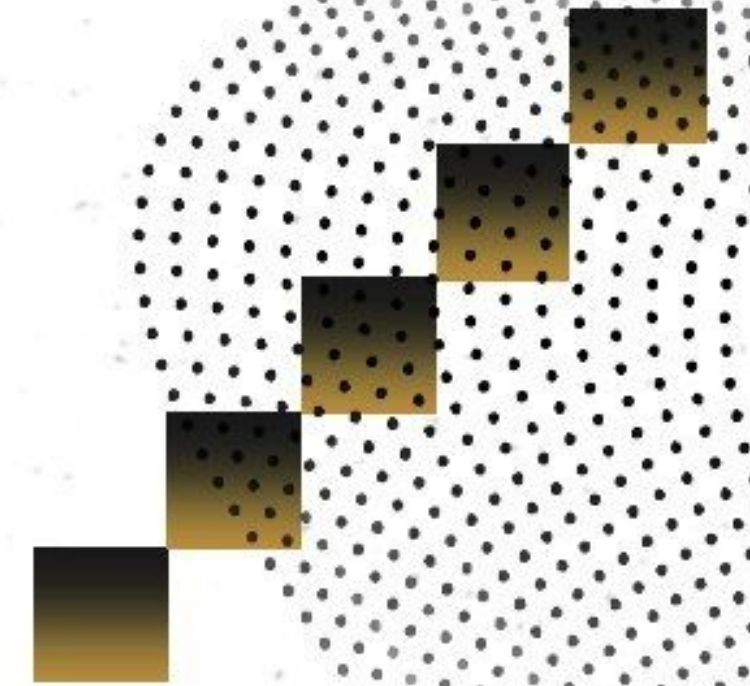
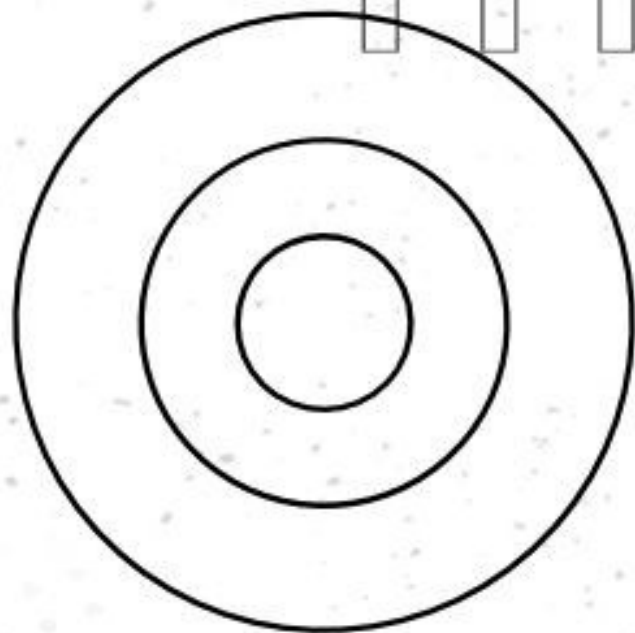
**“AN ARRAY STORES MULTIPLE VALUES IN A SINGLE VARIABLE...  
AND EACH VALUE HAS A POSITION CALLED AN INDEX.”**

**EXAMPLE:**

**[10, 20, 30, 40]**

**“10 IS AT INDEX 0, 20 AT INDEX 1...”**

**< T 0 >**





# DEFINE AN ARRAY

```
INT[] ARR = {10, 20, 30, 40};
```

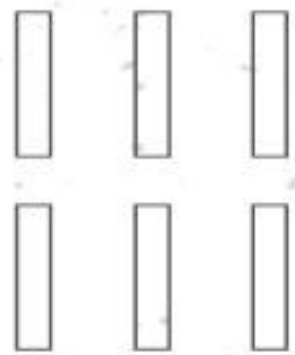
```
CONSOLE.WRITELINE(ARR[0]); // 10  
CONSOLE.WRITELINE(ARR[2]); // 30
```

< T O >

# INDEXING



**INDEX:**  
**ARR[0]**  
**ARR[1]**  
**ARR[2]**



< **T** **0** >





## CONTENT:

1D ARRAY

2D ARRAY (TABLE/MATRIX)

### 1D:

“SIMPLE LIST OF VALUES”

INT[] NUMBERS = {1, 2, 3};

# TYPES OF ARRAYS

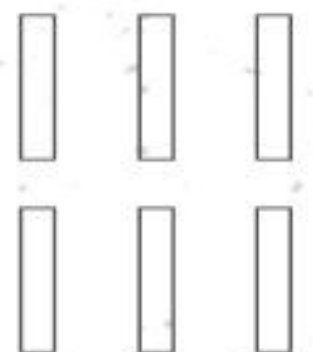
### 2D:

“ARRAY INSIDE ARRAY (LIKE A TABLE)”

INT[,] MATRIX = {  
{1, 2},  
{3, 4}  
};



# TYPES OF ARRAYS “V





What is **Array**?

**2 Dimensional**

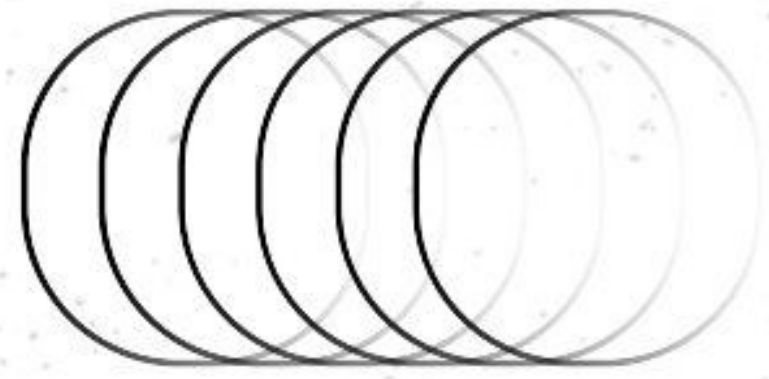
**Size 3 x 3**

# TYPES OF ARRAYS “VIDEO”

[  
[







# ARRAY OPERATIONS

**ACCESS → DIRECT**

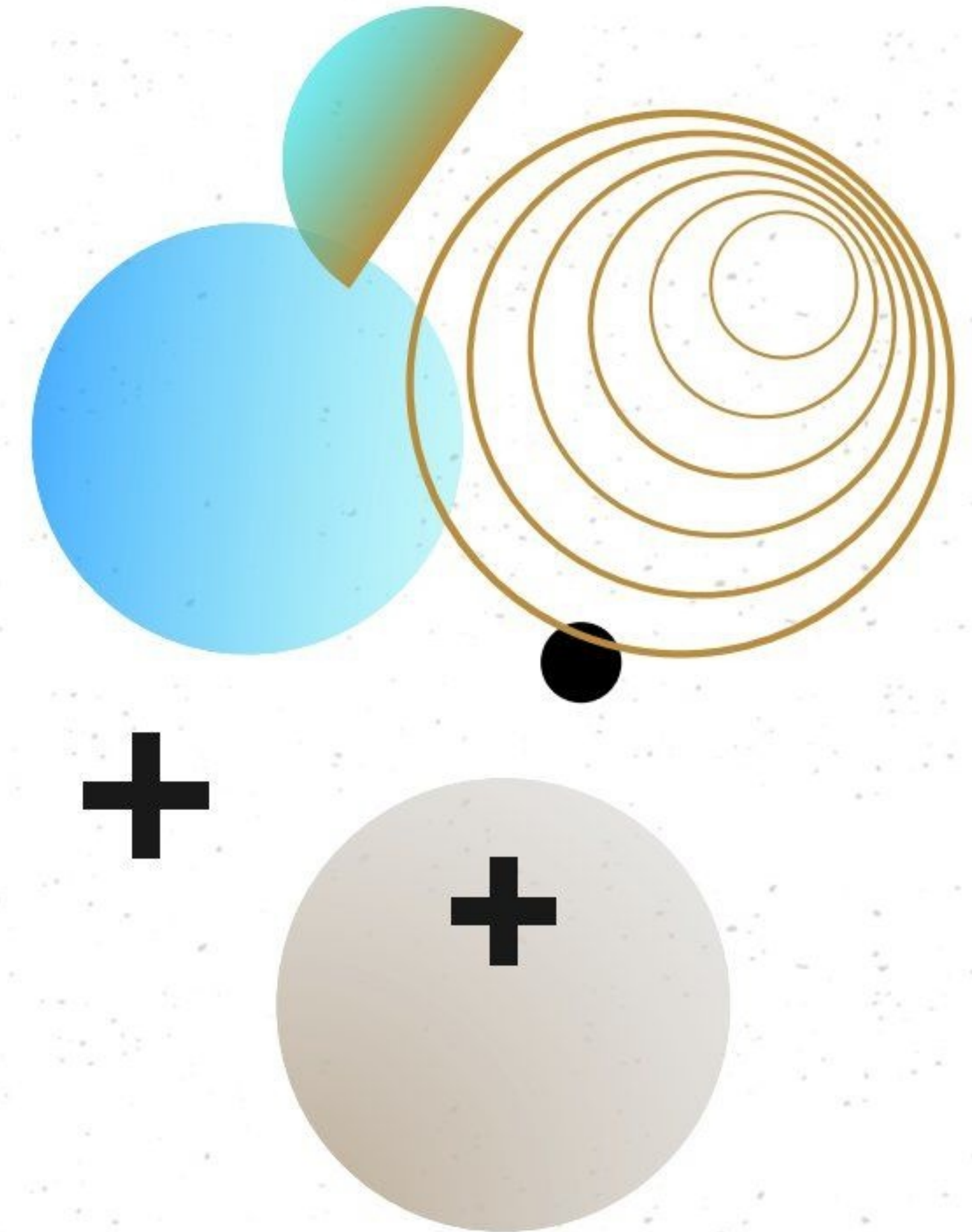
**SEARCH → CHECK ONE BY ONE**

**INSERT → SHIFT ELEMENTS**

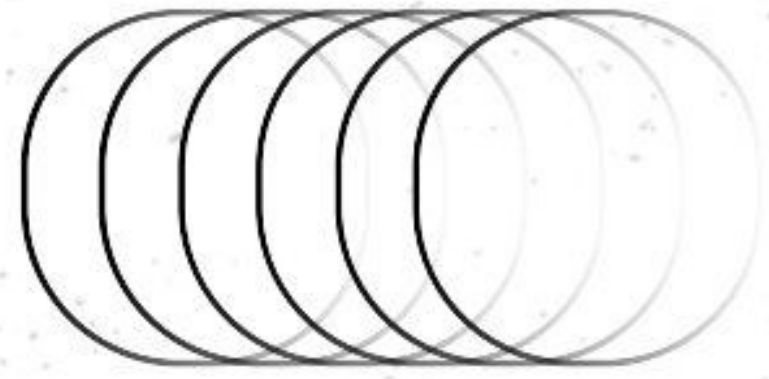
**DELETE → SHIFT ELEMENTS**



**< 0 1 >**







# SEARCHING IN ARRAYS

**EXAMPLE:**

**[10, 20, 30, 40]**

**FIND 30**

**PSEUDOCODE:**

**FOR EACH ELEMENT IN ARRAY:**

**IF ELEMENT == TARGET:**

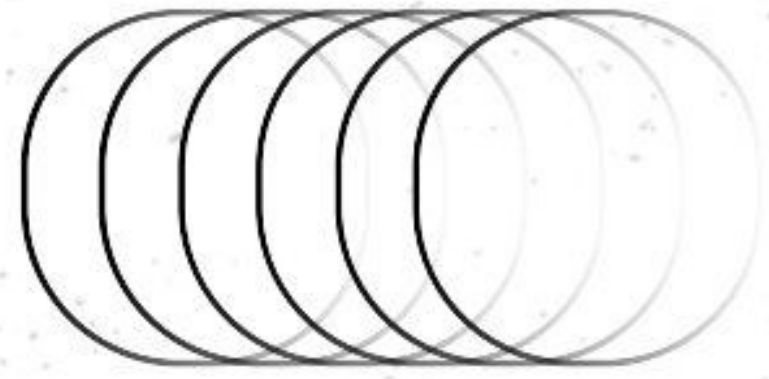
**RETURN INDEX**

**RETURN NOT FOUND**

**< 0 1 >**







# SEARCHING IN ARRAYS

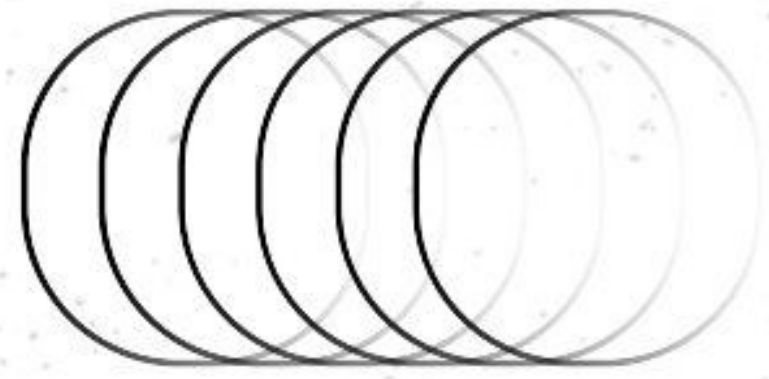
```
INT[] ARR = {10, 20, 30, 40};  
INT TARGET = 30;  
INT INDEX = -1;
```

```
FOR (INT I = 0; I < ARR.LENGTH; I++)  
{  
    IF (ARR[I] == TARGET)  
    {
```

< 0 1 >







# SEARCHING IN ARRAYS

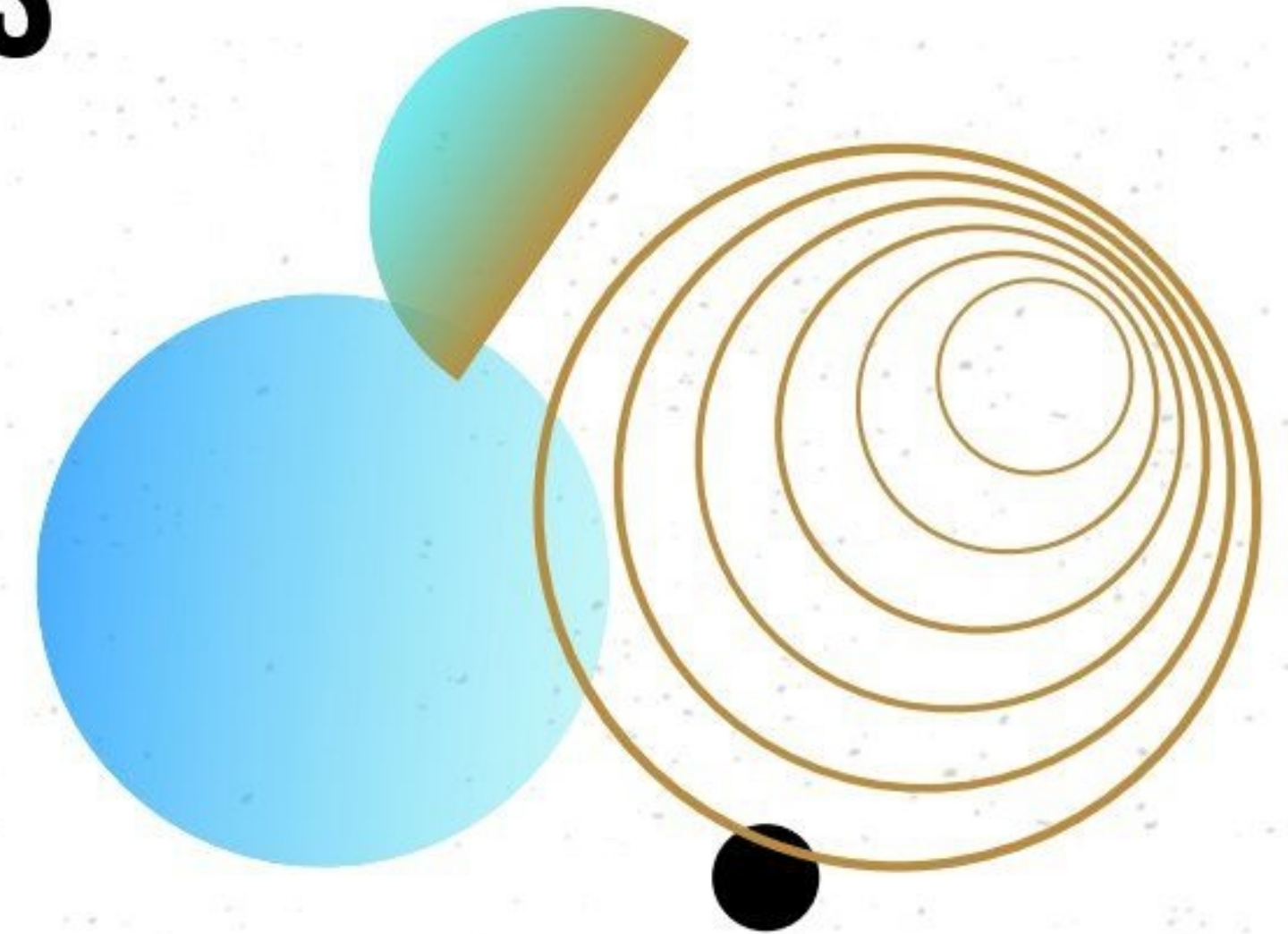
```
INDEX = I;  
    BREAK;  
}  
}
```

```
CONSOLE.WRITELINE(INDEX); //
```

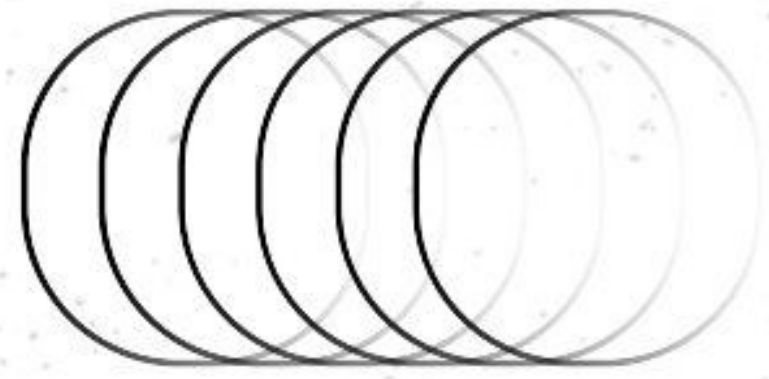
**OUTPUT: 2**

**CONNECT TO BIG-O:**

**“IN WORST CASE, WE CHECK ALL ELEMENTS  
→  $O(N)$ ”**







# ARRAY LIMITATIONS

**FIXED SIZE**

**INT[] ARR = NEW INT[5]; // FIXED SIZE**

**DIFFICULT INSERT/DELETE**

**INSERTING REQUIRES SHIFTING ELEMENTS**

**[10, 20, 30, 40]**

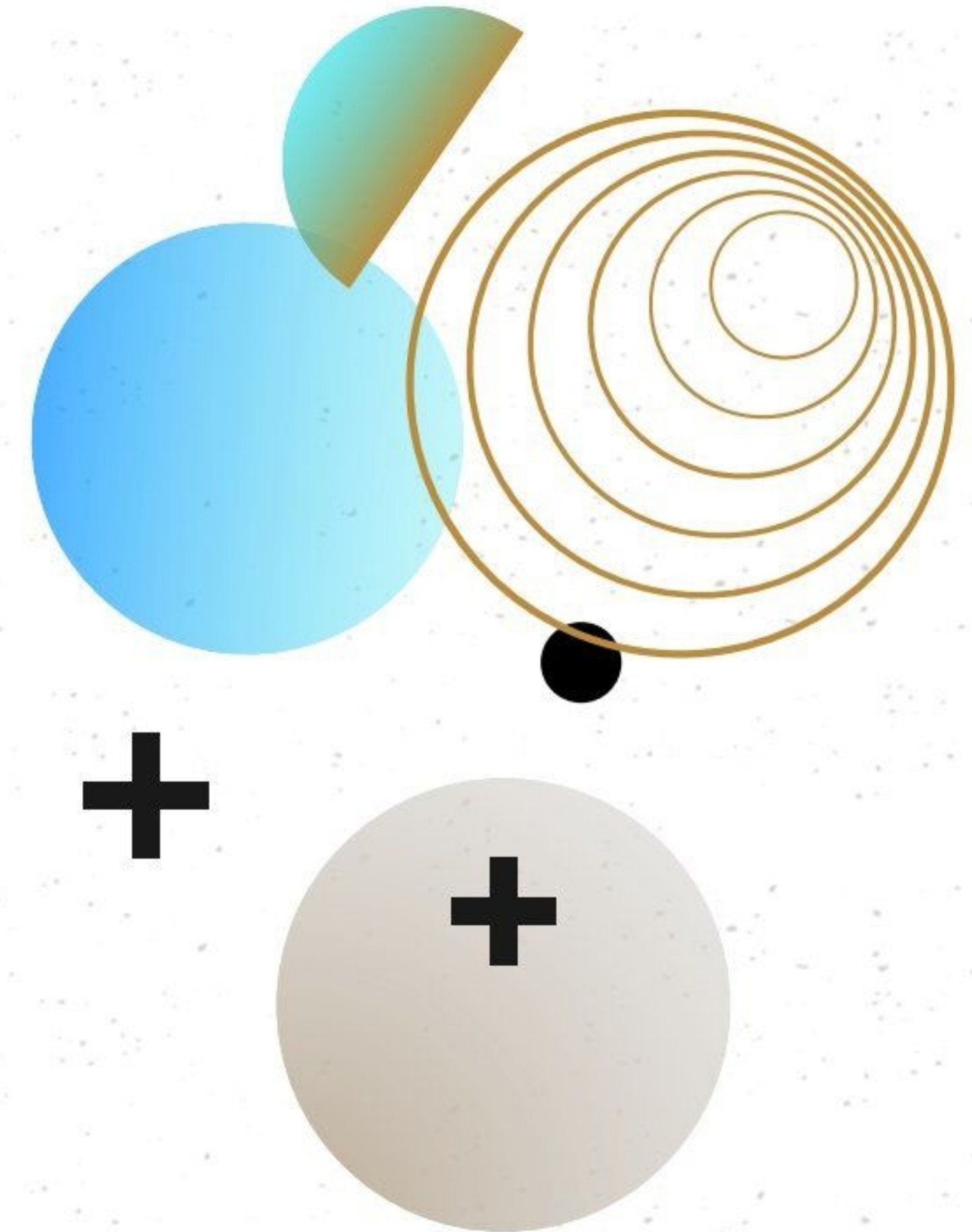
**INSERT 25 AT INDEX 2**

**> [10, 20, 25, 30, 40]**

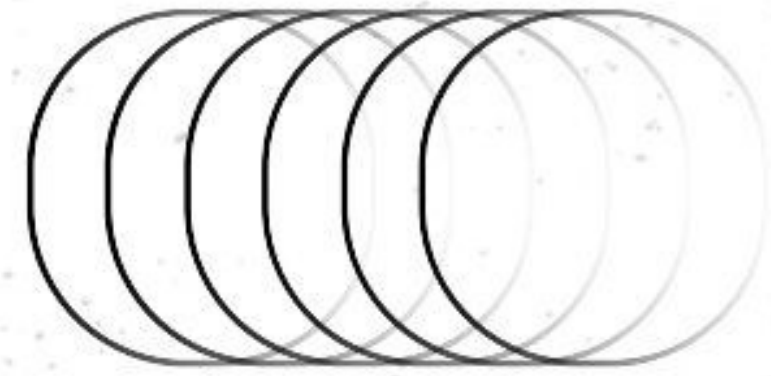
**NOT FLEXIBLE**



**< [ 0 ] [ 1 ] >**







# INTRODUCTION TO STACK

**A STACK IS A LINEAR STRUCTURE THAT FOLLOWS LIFO (LAST IN FIRST OUT)**



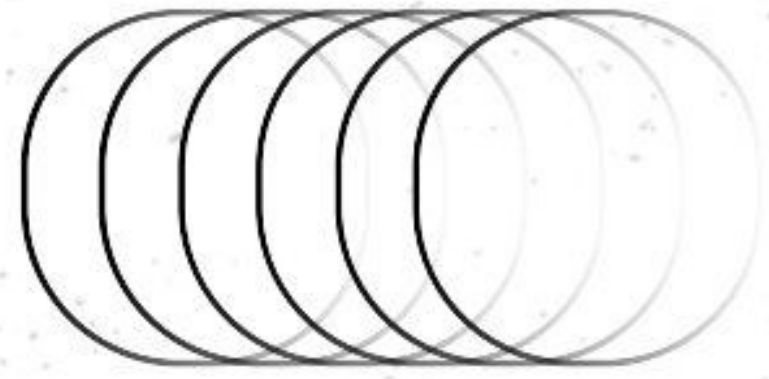
+

+



< 0 1 >





# STACK OPERATIONS

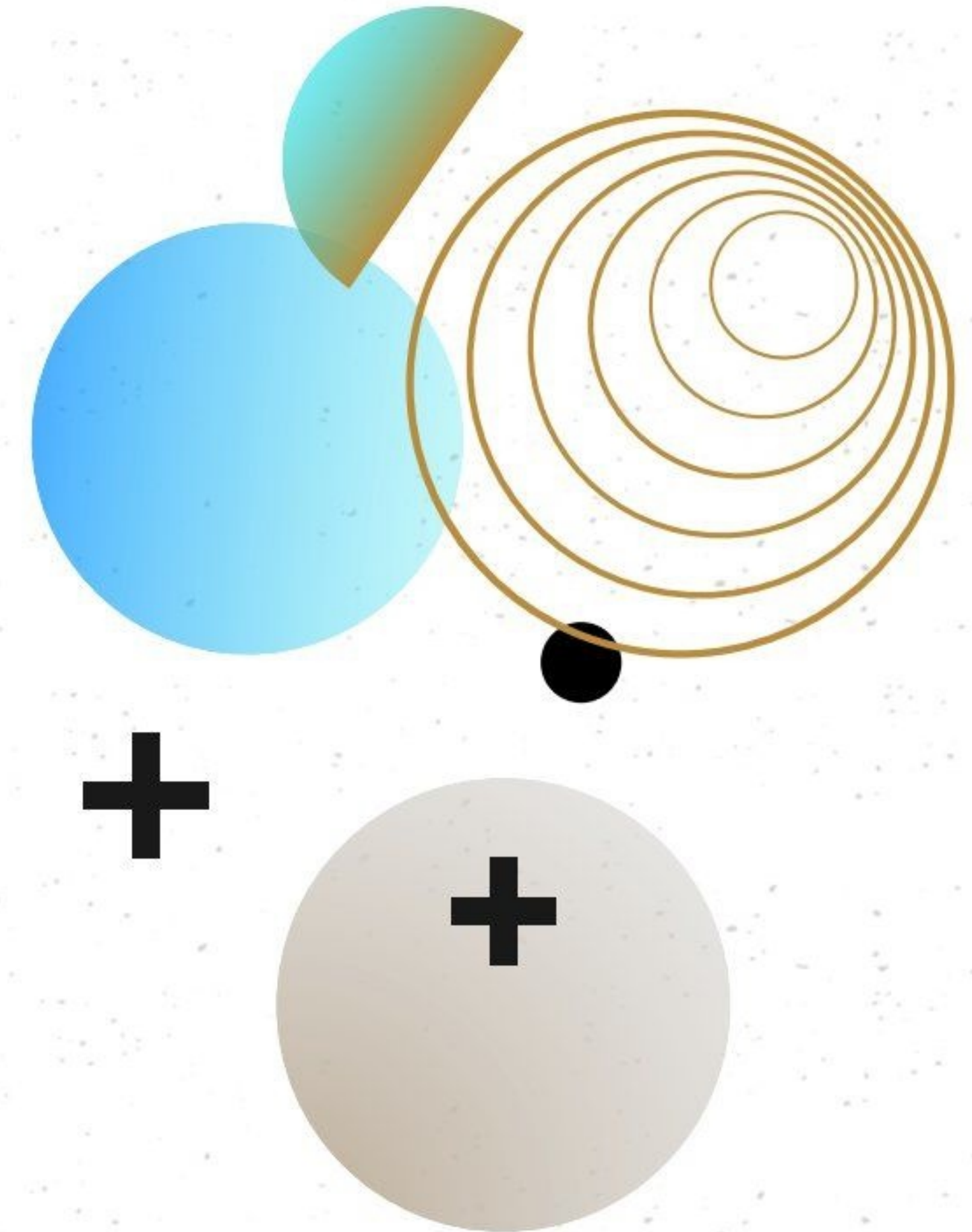
**PUSH → ADD**

**POP → REMOVE**

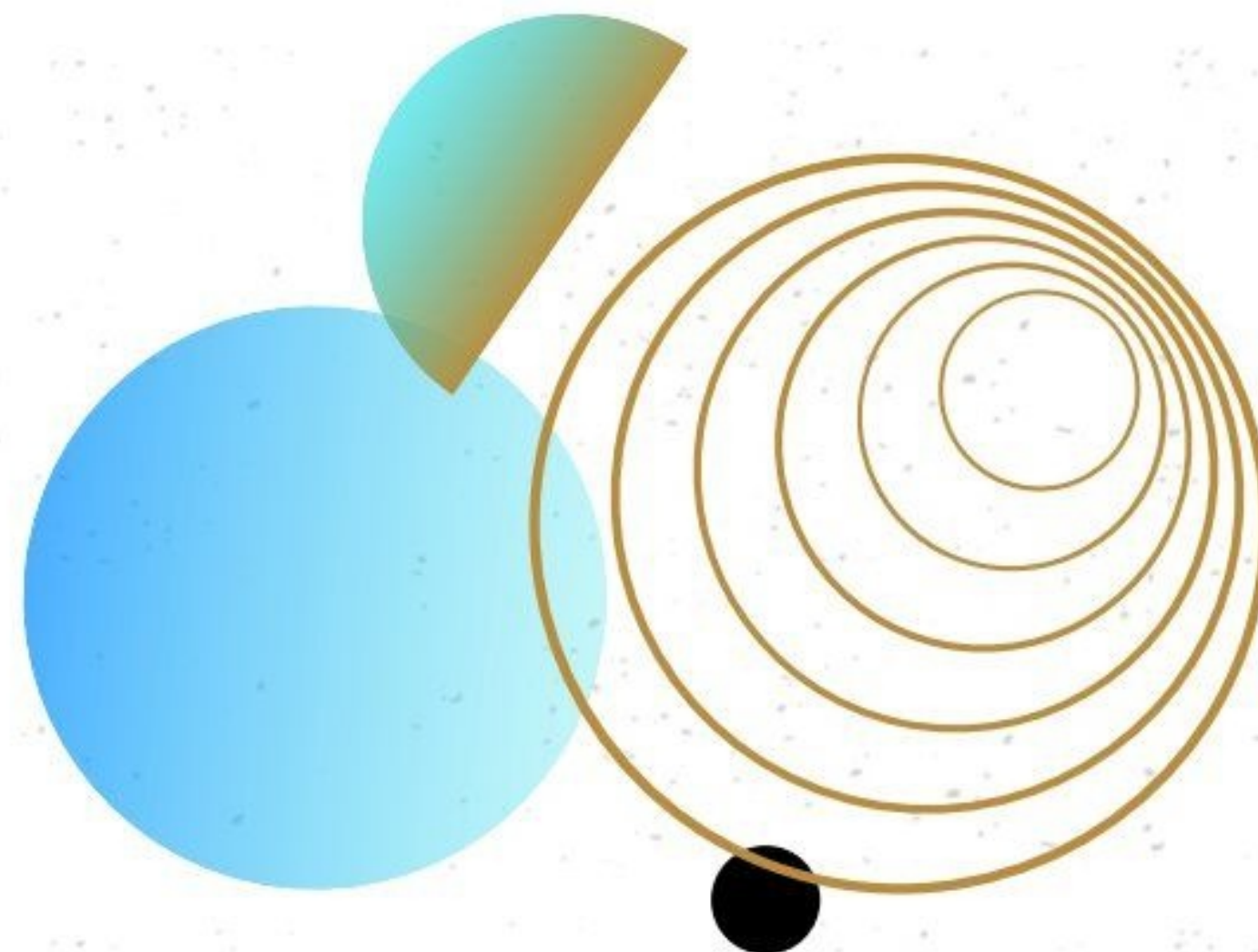
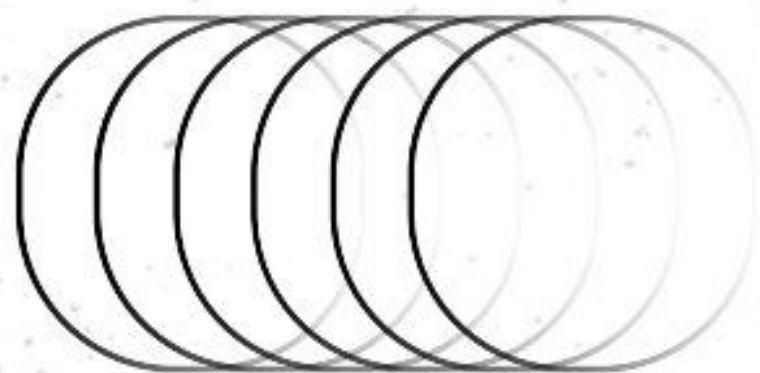
**PEEK → VIEW TOP**



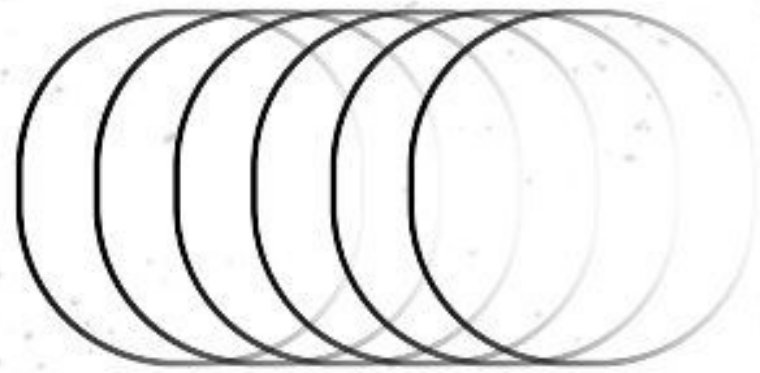
**< 0 1 >**





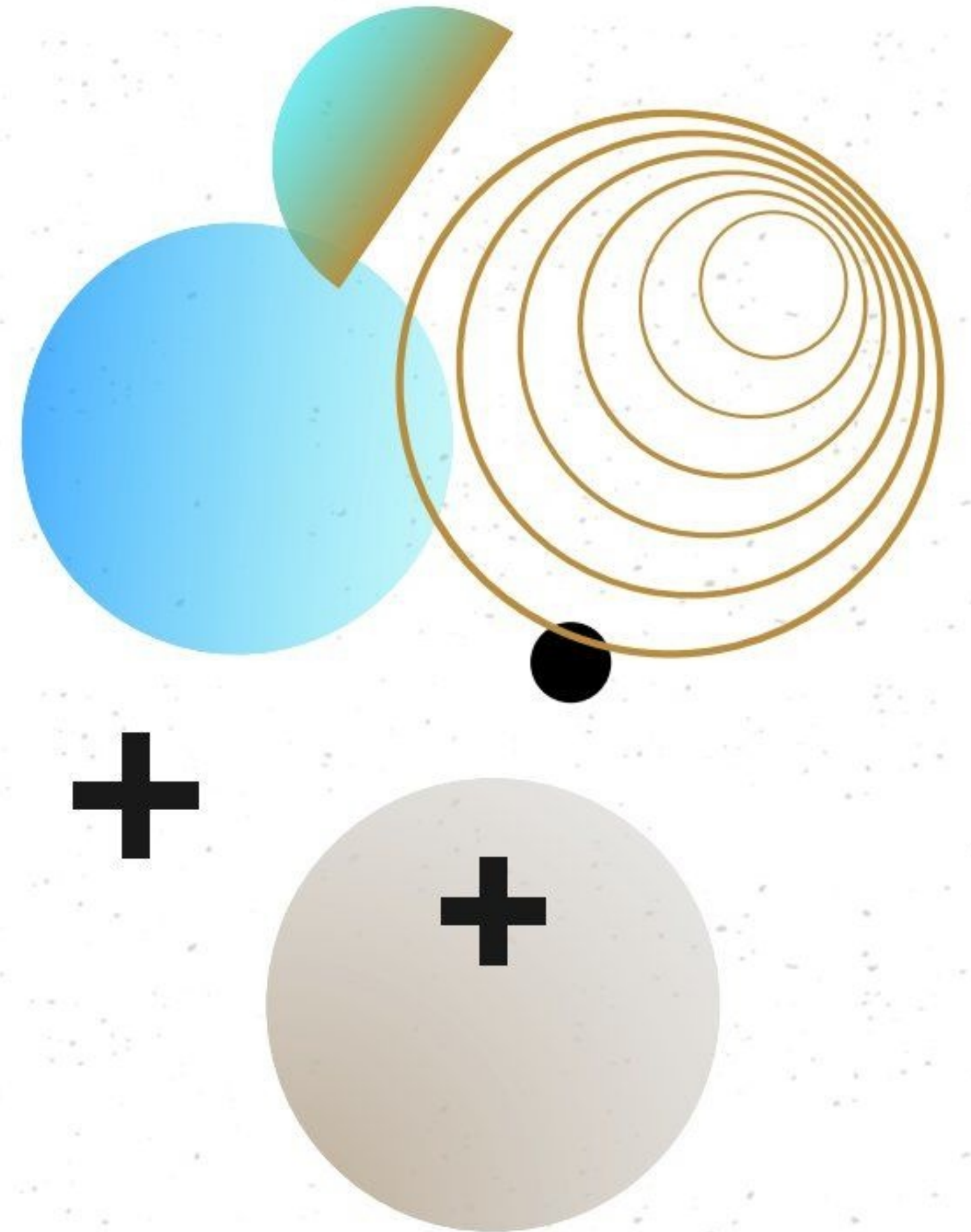




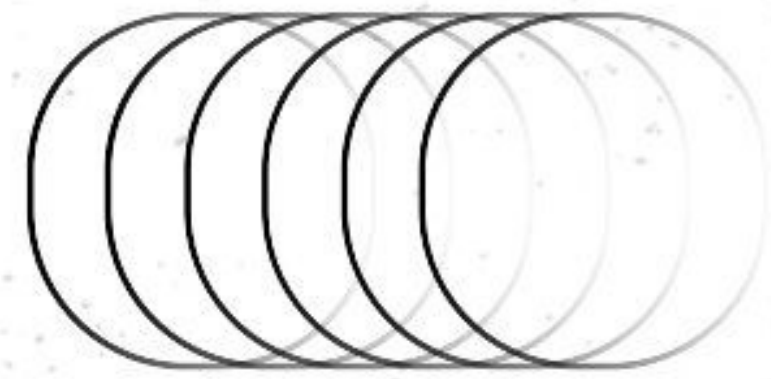


VilearnX

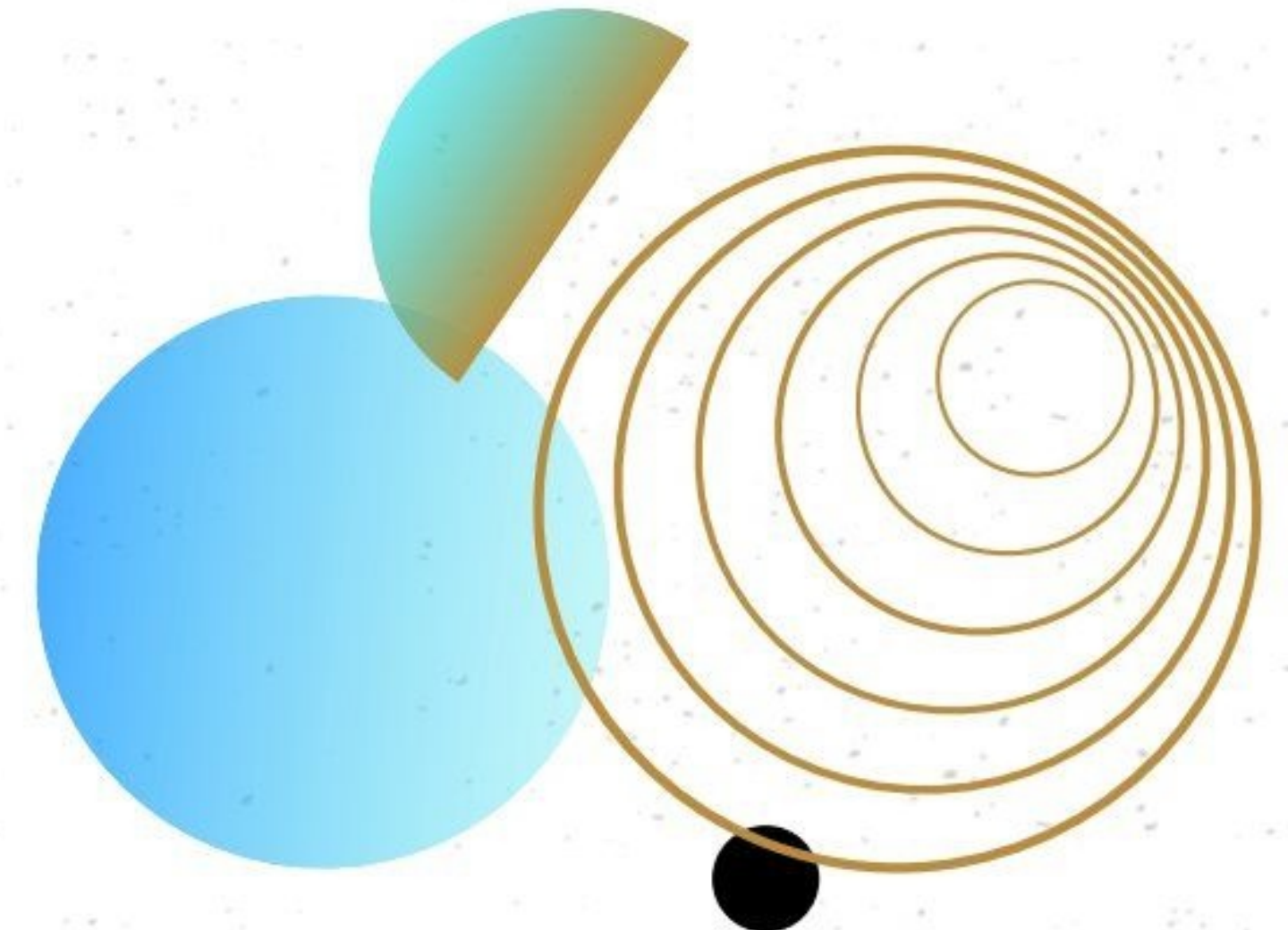
S







# STACK OPERATIONS



+

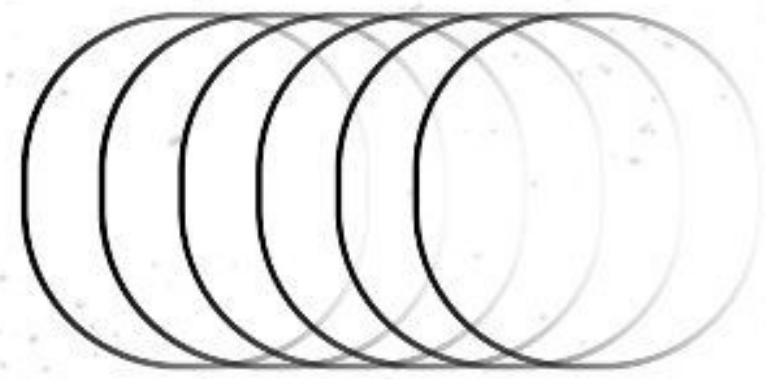
+

× × × ×  
× × × ×  
× × × ×  
× × × ×



< 0 1 >





# STACK OPERATIONS PSEUDOCODE

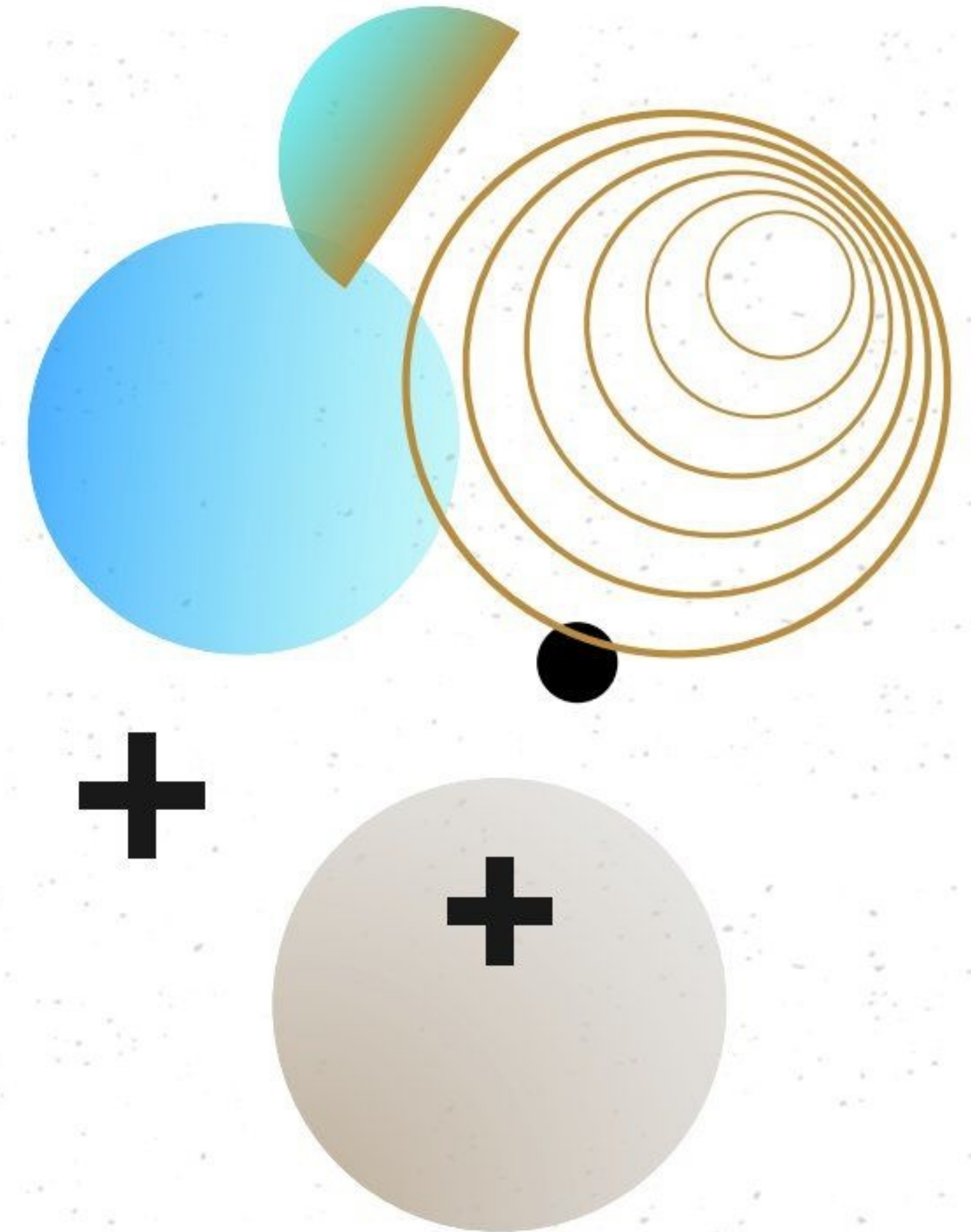
**PUSH(STACK, X):**  
ADD X TO TOP

**POP(STACK):**  
REMOVE TOP ELEMENT

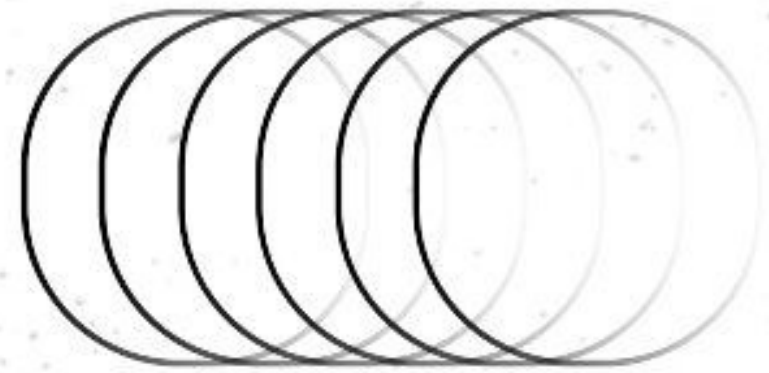
**PEEK(STACK):**  
RETURN TOP ELEMENT



< 0 1 >







# STACK OPERATIONS C#

```
STACK<INT> STACK = NEW  
STACK<INT>();
```

```
// PUSH
```

```
STACK.PUSH(10);  
STACK.PUSH(20);  
STACK.PUSH(30);
```



```
// PEEK
```

```
CONSOLE.WRITELINE(STACK.PEEK()); // 30
```

```
// POP
```

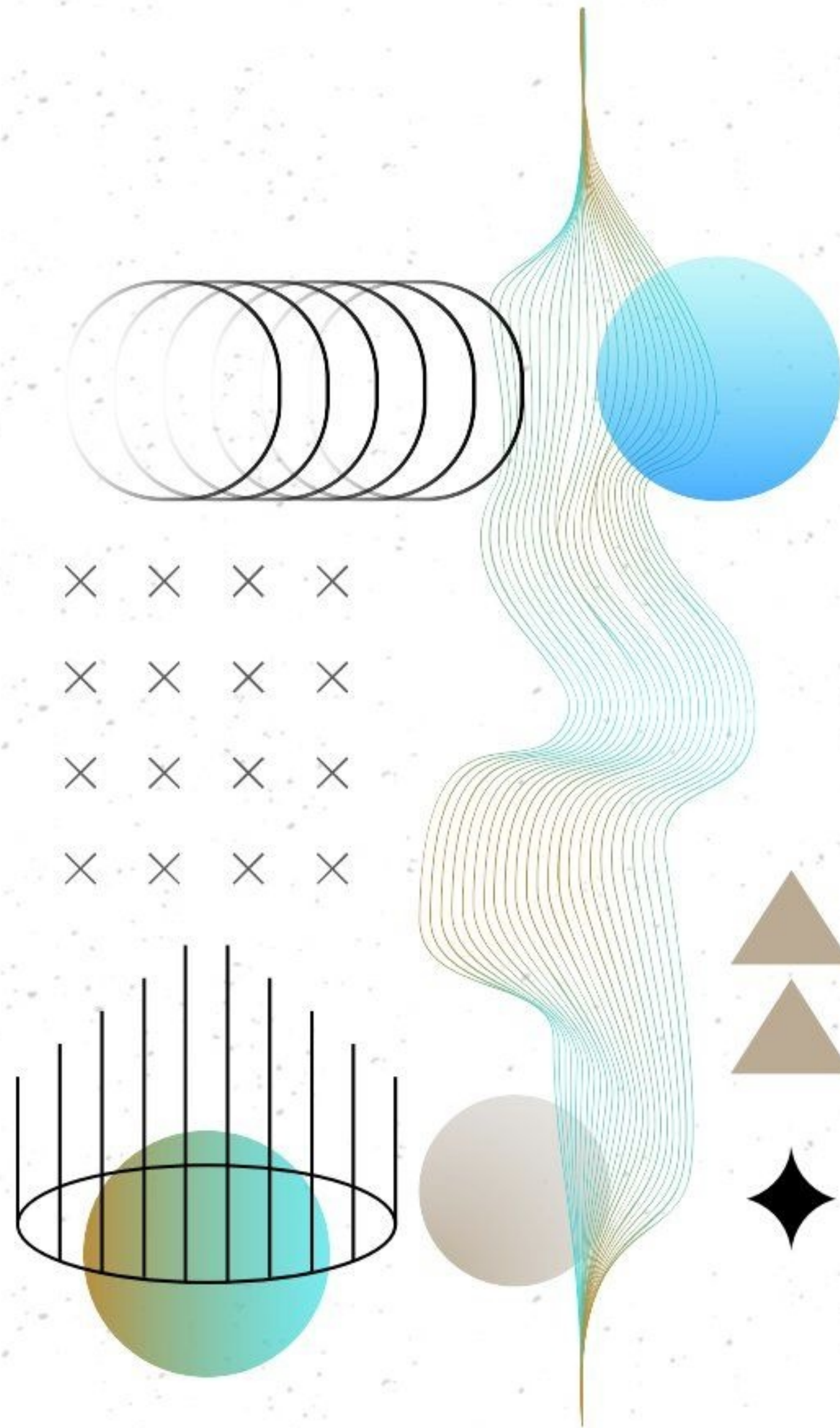
```
STACK.POP();  
CONSOLE.WRITELINE(STACK.PEEK()); // 20
```



< 0 1 >



# INTRODUCTION TO QUEUE

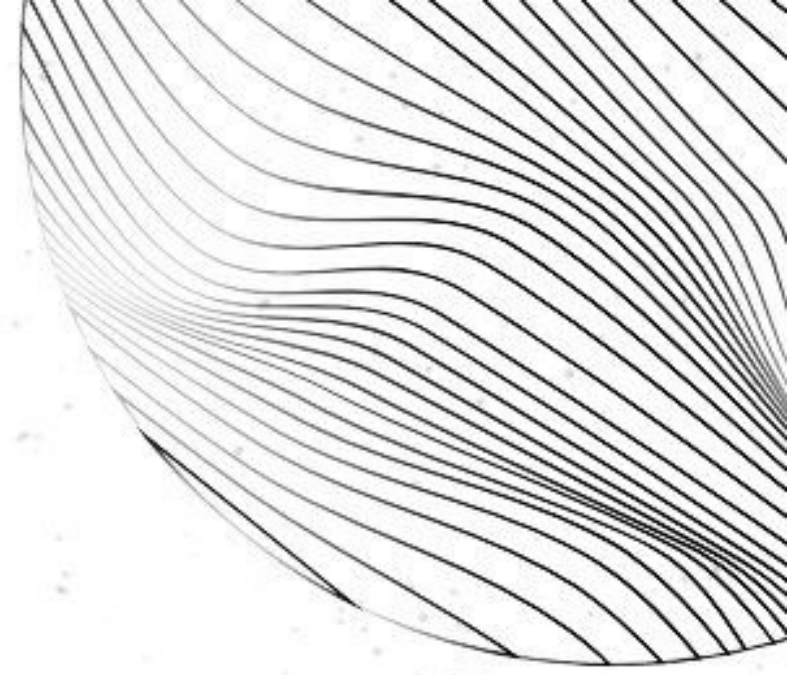


A Queue is a linear structure that follows FIFO (First In First Out)

“Unlike a stack, where the last element comes out first... a queue works like a real waiting line.”

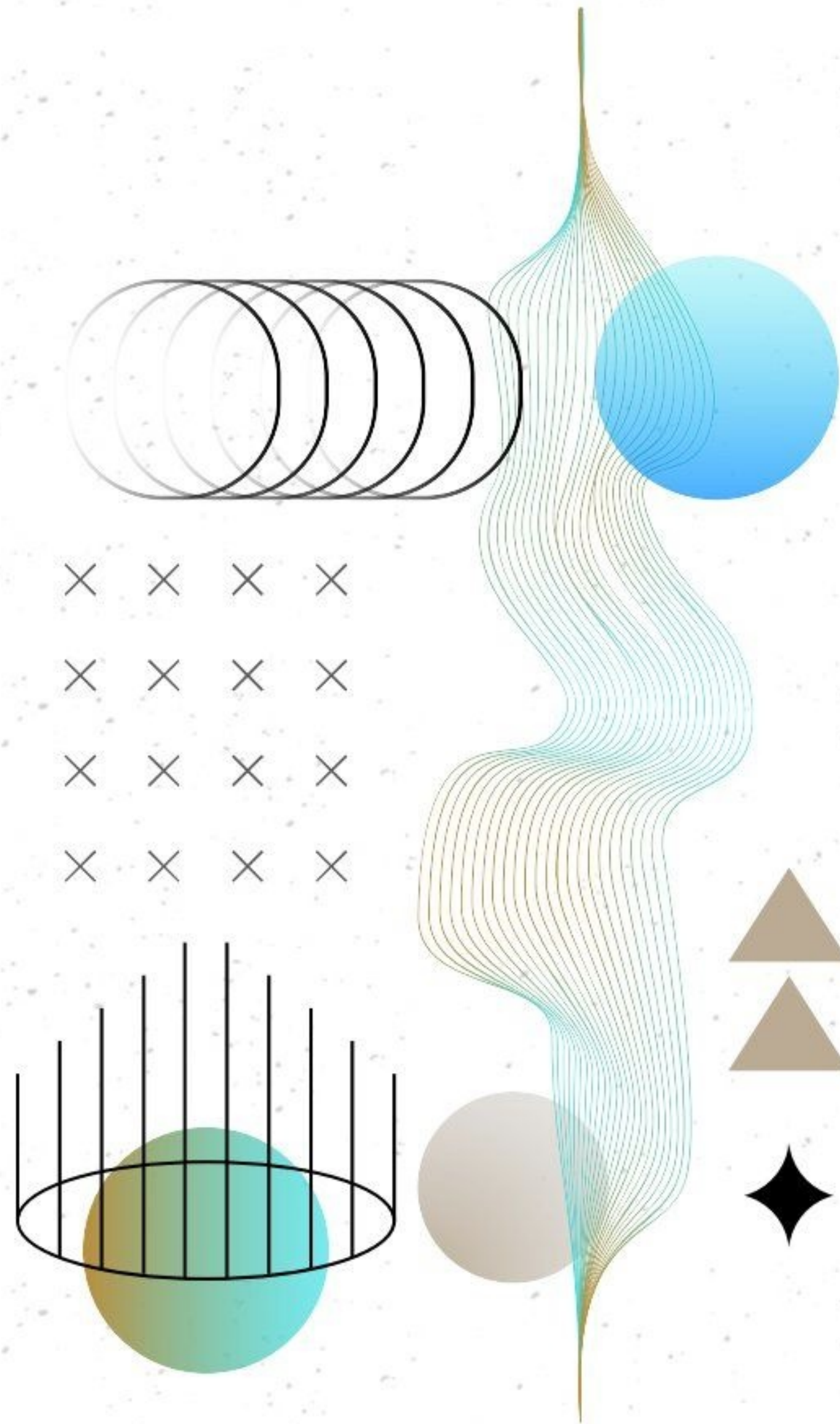


# INTRODUCTION TO QUEUE





# QUEUE OPERATIONS



enqueue → add  
dequeue → remove  
front → first element



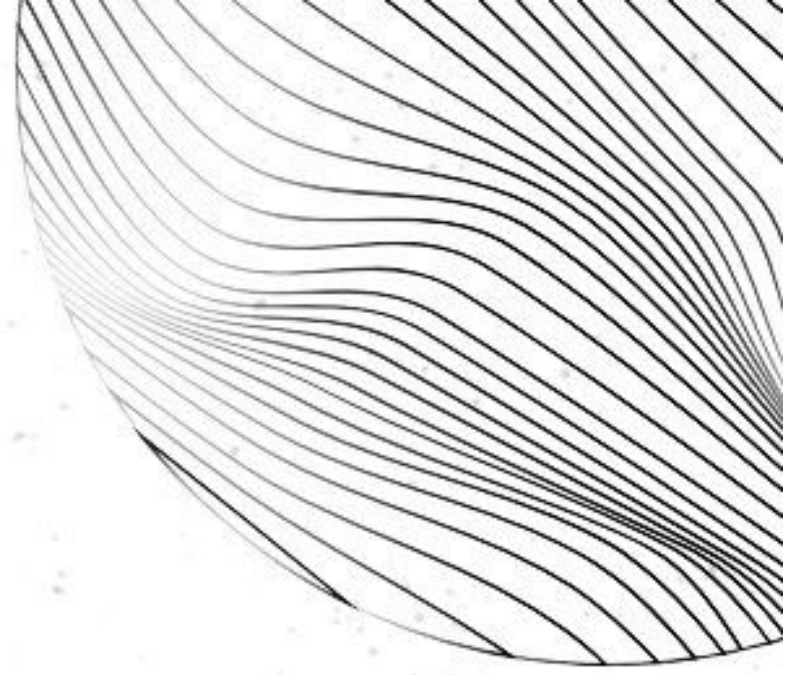
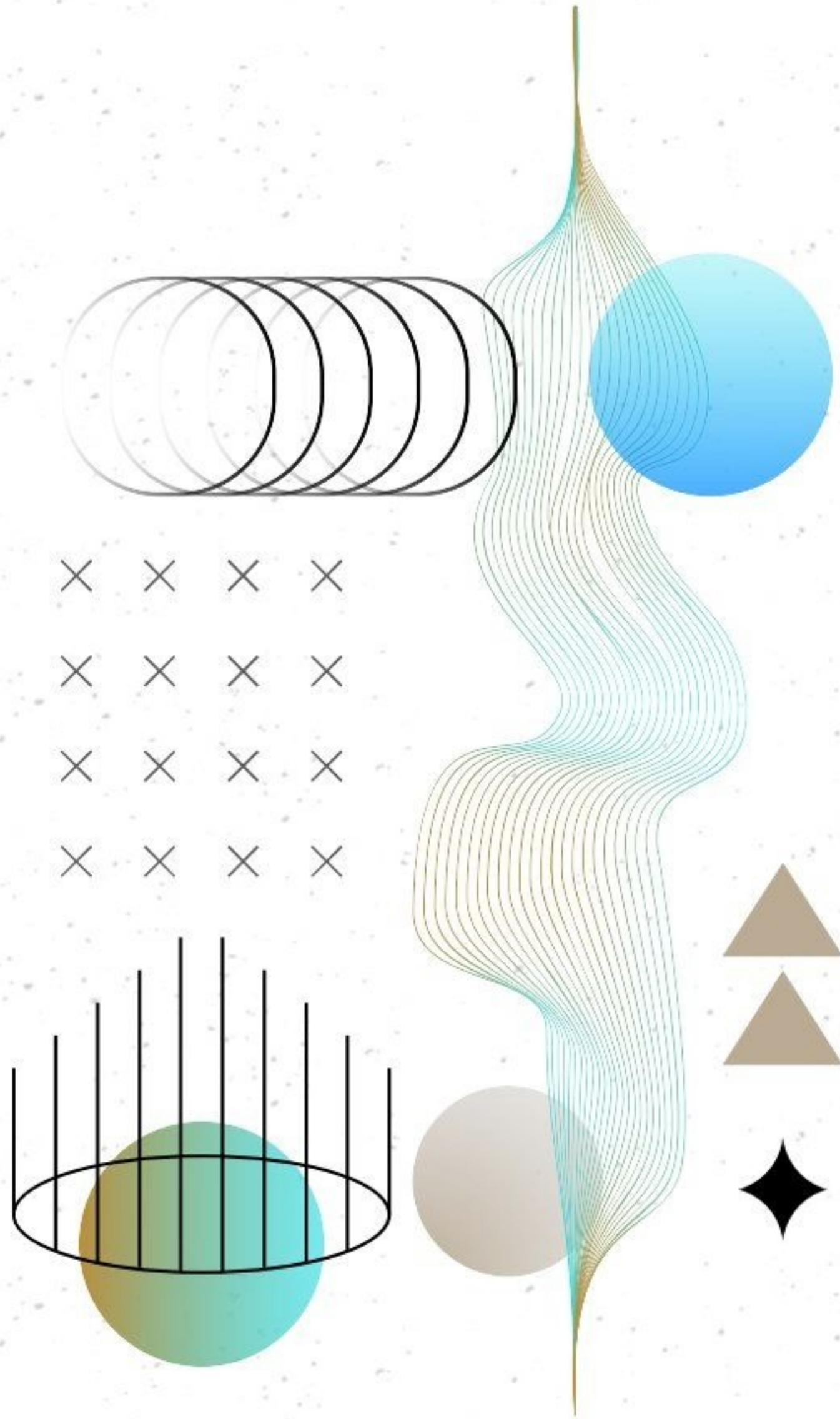


# QUEUE PSEUDOCODE

ENQUEUE(queue, x):  
add x to rear

DEQUEUE(queue):  
remove front element

FRONT(queue):  
return first element





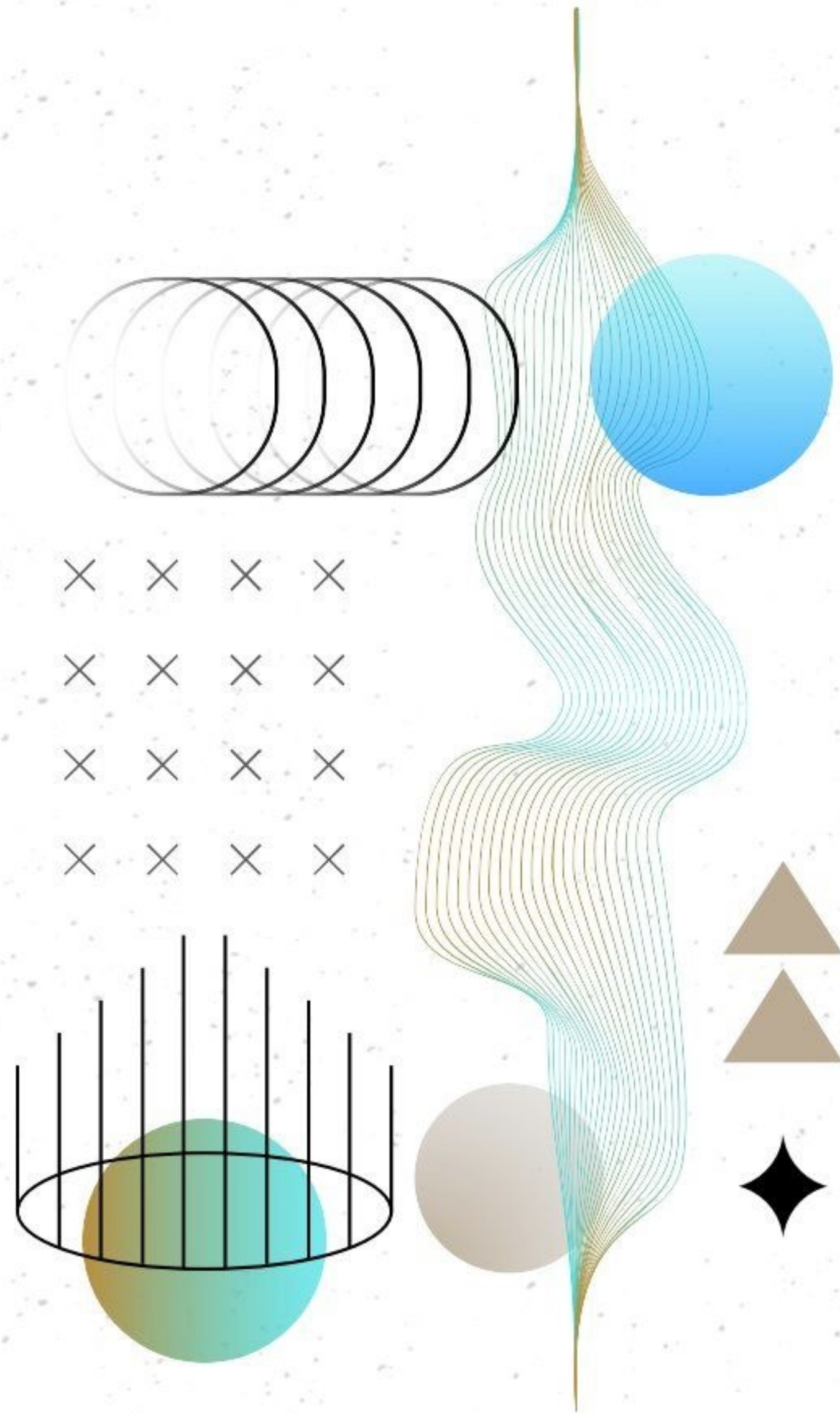
# QUEUE C# IMPLEMENTATION

```
Queue<int> queue = new  
Queue<int>();
```

```
// ENQUEUE  
queue.Enqueue(10);  
queue.Enqueue(20);  
queue.Enqueue(30);
```



# QUEUE C# IMPLEMENTATION



```
// FRONT  
Console.WriteLine(queue.Peek());  
// 10
```

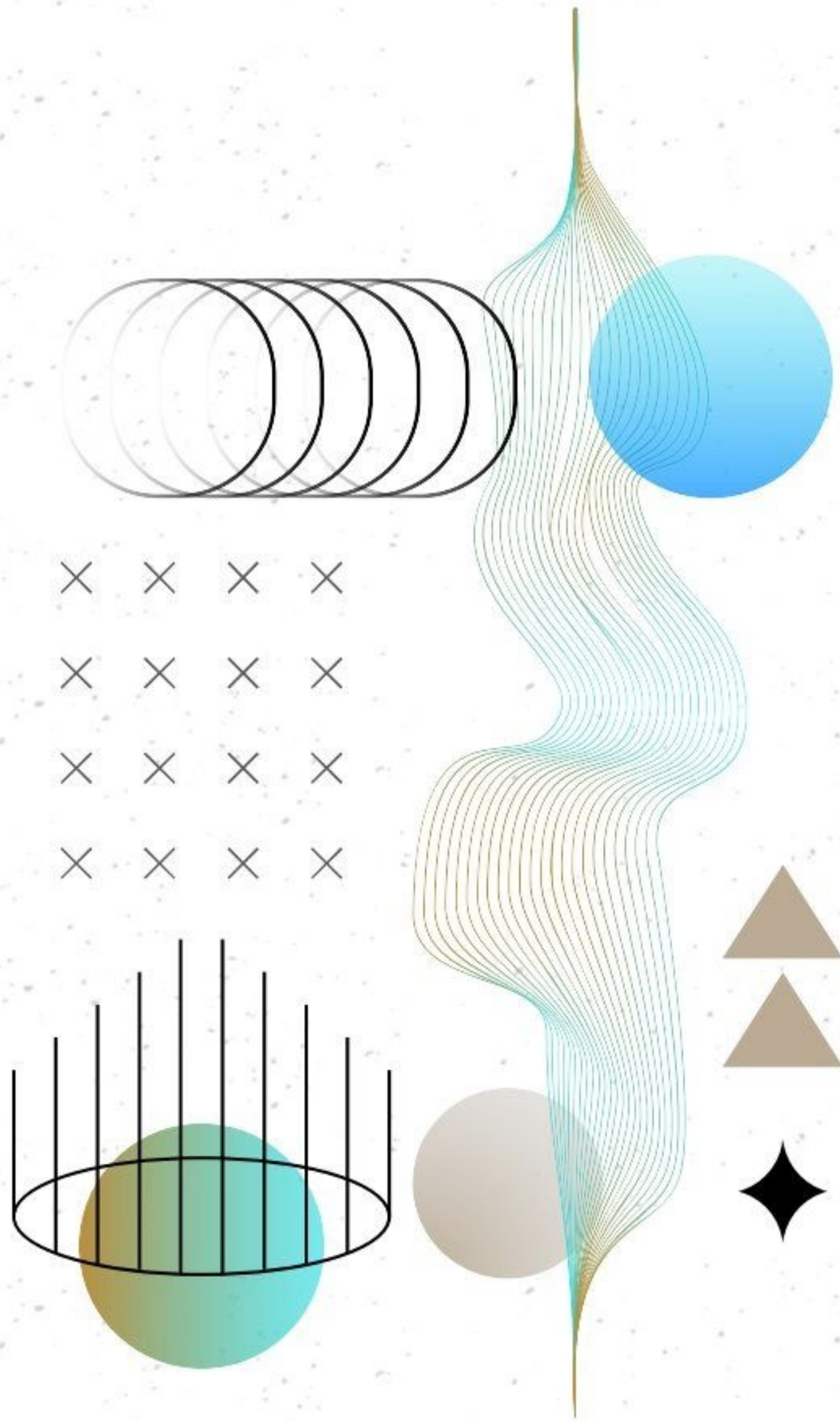
```
// DEQUEUE  
queue.Dequeue();  
Console.WriteLine(queue.Peek());  
// 20
```





# QUEUE TYPES

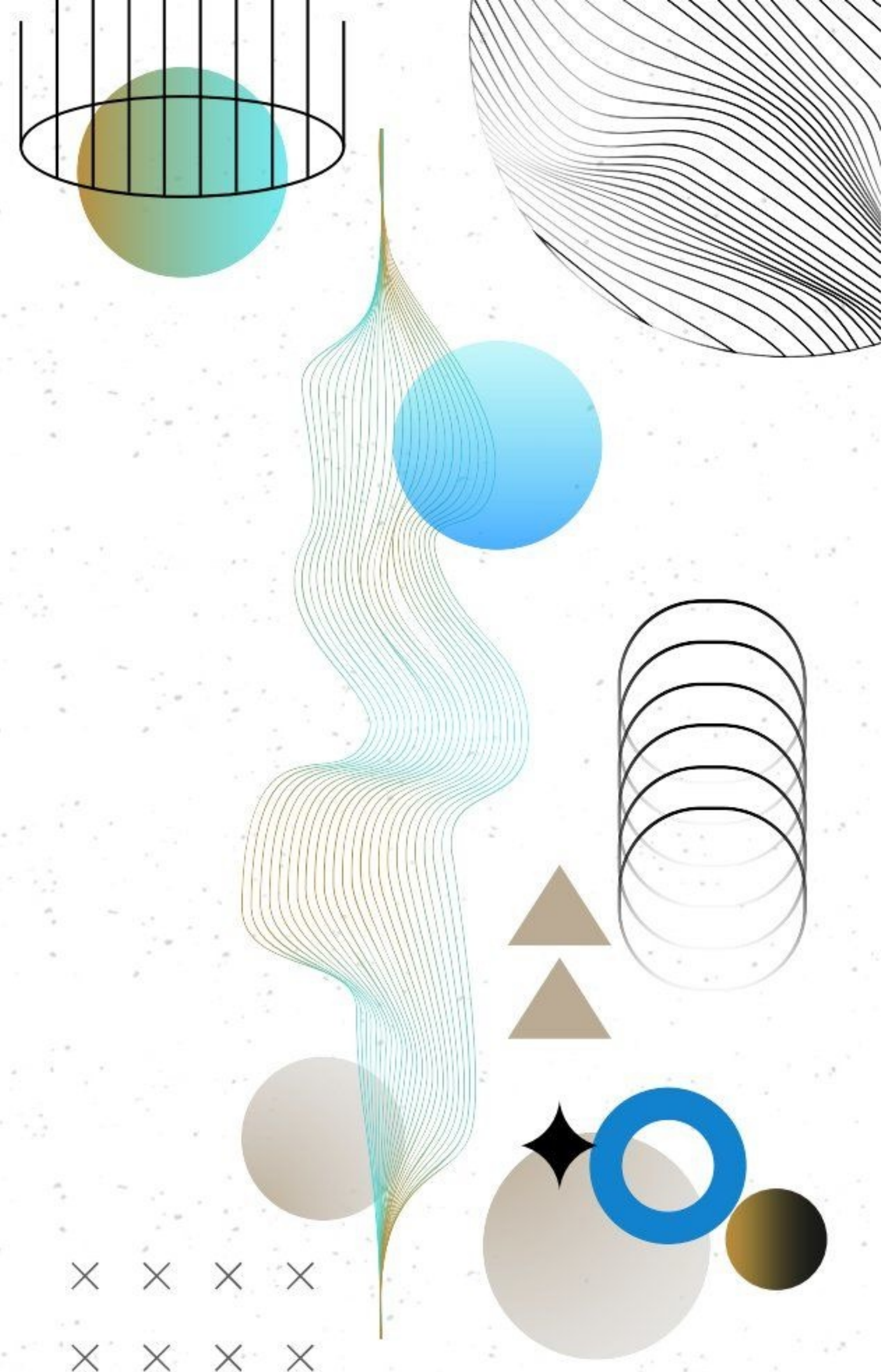
Simple Queue  
Circular Queue (mention only)  
Priority Queue (mention only)





# QUEUE PERFORMANCE

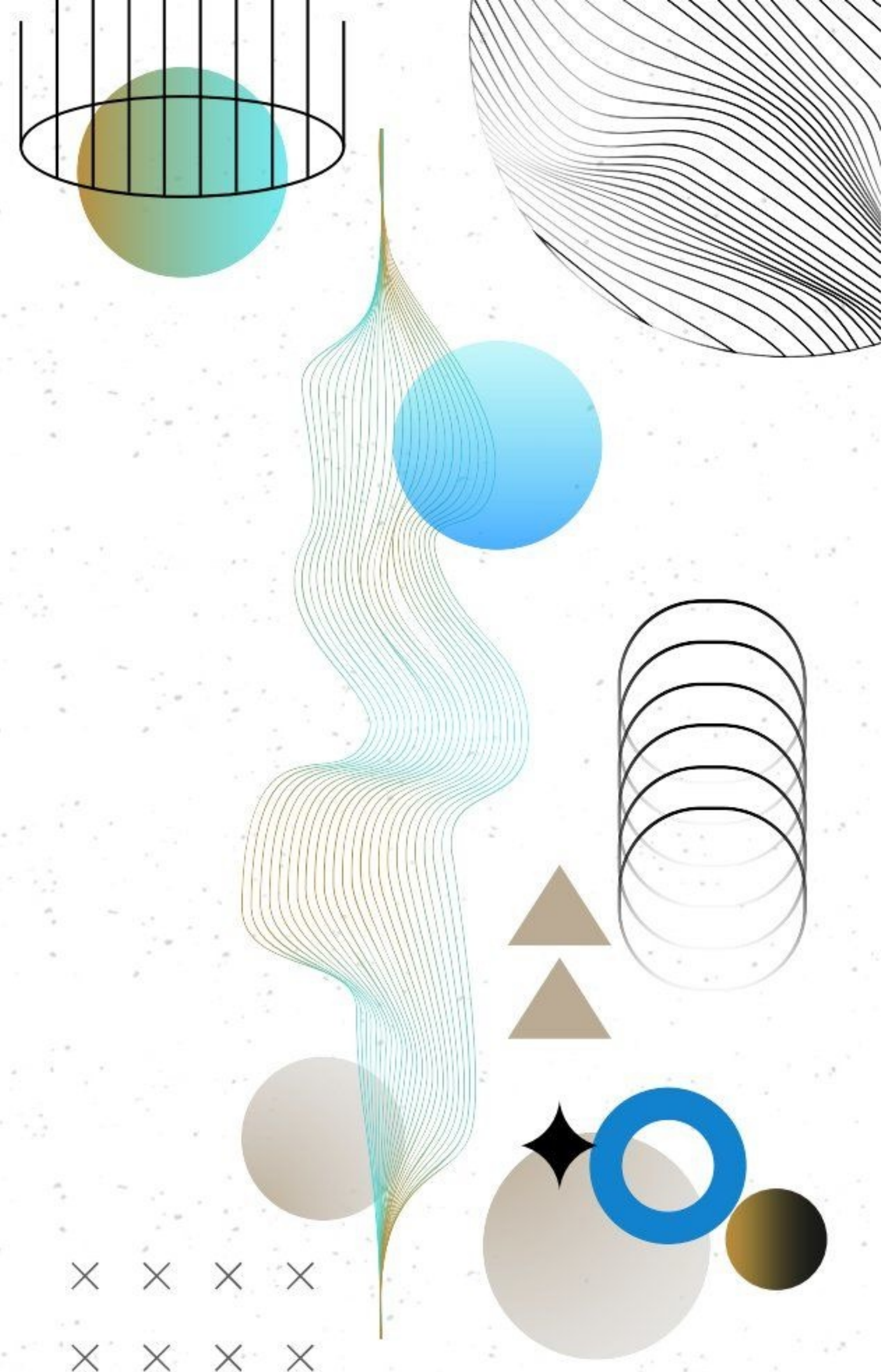
- Enqueue  $\rightarrow O(1)$
- Dequeue  $\rightarrow O(1)$





# QUEUE APPLICATIONS

- Scheduling systems
- Operating systems
- BFS (Graph traversal)





# CONTIGUOUS MEMORY

**The Concept:** When you create an array, the computer finds a **single, unbroken block** of memory.

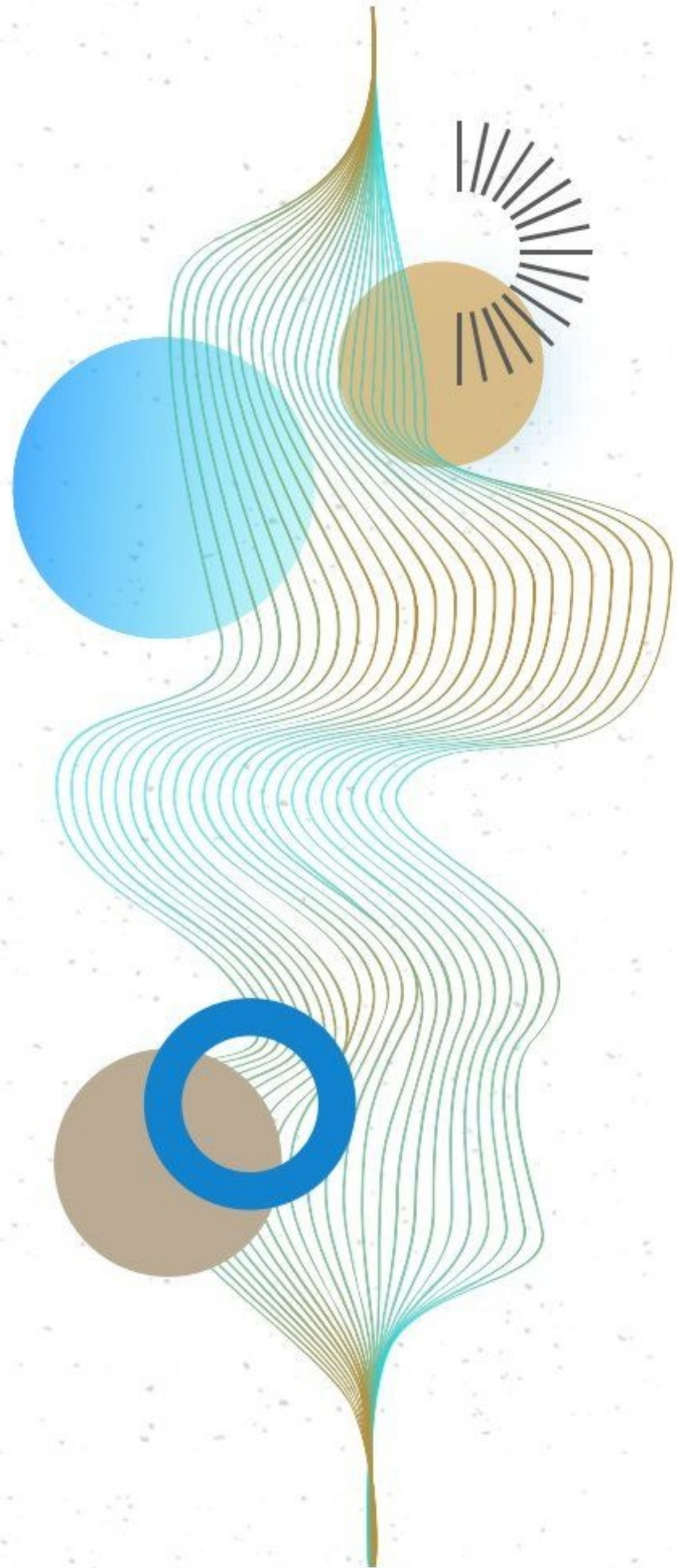
**The Visual:** Imagine a train. Every carriage is attached to the next. You can't have a carriage floating in another city.

**The Benefit:** Because they are side-by-side, the computer can calculate exactly where `index[4]` is without looking at index 1, 2, or 3.

**Why it matters:** This is why Arrays have  **$O(1)$  Access Time**. It's just a math calculation!







**THANK**

**YOU !**

